Note:

For all the questions candidates need to write c Code.

Reference Microcontrollwe is STM32F4.

---

1. **Basic Timer Configuration**:
   How would you configure **Timer 2** in the STM32F4 to generate an interrupt every 1 second using a 16 MHz clock source? Provide code to initialize the timer and set up the interrupt.
2. **Cascading Timers**:
   Explain how you can use two timers in cascade mode to achieve a longer time interval than what a single timer can provide on STM32F4. Write code to cascade **Timer 3** and **Timer 4** to run as free flow clock. Write a code to generate a periodic timer callback of 10 second.
3. **Circular Buffer Implementation for UART**:
   Implement a **circular buffer** in C to handle UART data reception in a real-time system. Write code to read data from the buffer and process it, ensuring that no data is lost even when new data arrives rapidly.
4. **Checksum Calculation for Data Integrity:**
   Write a C program that calculates a **checksum** for a byte-packed array of data before transmitting it over UART. Explain how this checksum can be used on the receiving end to verify the integrity of the received data.
5. **Parsing Received Data from UART:**
   Write a function in C to parse incoming data over UART that has been received as a byte array. Assume the data represents a packed structure containing a sensor ID, a timestamp, and a reading value.
6. **Array of Structures for Data Handling:**
   Implement an array of structures in C that holds sensor data (temperature, pressure, humidity). Write a function to serialize this array into a byte stream suitable for transmission over UART.
7. **Finite State Machine (FSM) Basics:**
   Create a **finite state machine (FSM)** in C for a **simple vending machine**. The machine should have states like "Idle," "Selecting Product," "Processing Payment," and "Dispensing Product." Provide the state transition logic based on user inputs.
8. **Super Loop Implementation:**
   Explain the concept of a **super loop** in C programming. Write a simple C program that continuously monitors three different flags in a super loop and performs an action when a specific flag is set.

Write a C program that transfers a file from a **client** to a **server** using UDP communication. The program should take the following command-line arguments:

1. The **file name** to be transferred.
2. The **file location** (directory path) where the file is located on the client.
3. The **storage location** (directory path) where the file should be saved on the server.

The program should be structured as follows:

- Implement two separate programs: one for the **client** and one for the **server**.
- The **client program** should read the file from the specified location, break it into smaller packets, and send these packets to the server using UDP.
- The **server program** should listen for incoming packets, reconstruct the file from the received packets, and store it in the specified storage location.

Additional Requirements:

1. **Error Handling**: Include error handling for cases like file not found, network errors, packet loss, and file write errors.
2. **Data Acknowledgment**: Implement a simple acknowledgment mechanism to confirm that each packet is received correctly by the server.
3. **Packet Sequencing**: Use packet sequencing to ensure that the server can correctly reconstruct the file even if packets arrive out of order.
4. **Timeout and Retransmission**: Implement a timeout and retransmission mechanism in case an acknowledgment for a packet is not received by the client.